#HashEx
BLOCKCHAIN SECURITY

EVEDEX Arbitrum

smart contracts final audit report

August 2026

hashex.org

contact@hashex.org

Contents

1. Disclaimer 3

2. Overview 4

3. Project centralization risks 6

4. Found issues 9

5. Contracts 11

6. Conclusion 17

Appendix A. Issues' severity classification 18

Appendix B. Issue status description 19

Appendix C. List of examined issue types 20

Appendix D. Centralization risks classification 21

1. Disclaimer

This is a limited report on our findings based on our analysis, in accordance with good industry practice at the date of this report, in relation to cybersecurity vulnerabilities and issues in the framework and algorithms based on smart contracts, the details of which are set out in this report. In order to get a full view of our analysis, it is crucial for you to read the full report. While we have done our best in conducting our analysis and producing this report, it is important to note that you should not rely on this report and cannot claim against us on the basis of what it says or doesn't say, or how we produced it, and it is important for you to conduct your own independent investigations before making any decisions. We go into more detail on this in the disclaimer below – please make sure to read it in full.

By reading this report or any part of it, you agree to the terms of this disclaimer. If you do not agree to the terms, then please immediately cease reading this report, and delete and destroy any and all copies of this report downloaded and/or printed by you. This report is provided for information purposes only and on a non-reliance basis and does not constitute investment advice. No one shall have any right to rely on the report or its contents, and HashEx and its affiliates (including holding companies, shareholders, subsidiaries, employees, directors, officers, and other representatives) (HashEx) owe no duty of care towards you or any other person, nor does HashEx make any warranty or representation to any person on the accuracy or completeness of the report. The report is provided "as is", without any conditions, warranties, or other terms of any kind except as set out in this disclaimer, and HashEx hereby excludes all representations, warranties, conditions, and other terms (including, without limitation, the warranties implied by law of satisfactory quality, fitness for purpose and the use of reasonable care and skill) which, but for this clause, might have effect in relation to the report. Except and only to the extent that it is prohibited by law, HashEx hereby excludes all liability and responsibility, and neither you nor any other person shall have any claim against HashEx, for any amount or kind of loss or damage that may result to you or any other person (including without limitation, any direct, indirect, special, punitive, consequential or pure economic loss or damages, or any loss of income, profits, goodwill, data, contracts, use of money, or business interruption, and whether in delict, tort (including without limitation negligence), contract, breach of statutory duty, misrepresentation (whether innocent or negligent) or otherwise under any claim of any nature whatsoever in any jurisdiction) in any way arising from or connected with this report and the use, inability to use or the results of the use of this report, and any reliance on this report. The analysis of the security is purely based on the smart contracts alone. No applications or operations were reviewed for security. No product code has been reviewed. HashEx owns all copyright rights to the text, images, photographs, and other content provided in the following document. When using or sharing partly or in full, third parties must provide a direct link to the original document mentioning the author (hashex.org).

2. Overview

HashEx was commissioned by the EVEDEX team to perform an audit of their smart contracts. The audit was conducted between 11/08/2026 and 12/08/2026.

The purpose of this audit was to achieve the following:

- Identify potential security issues with smart contracts
- Formally check the logic behind given smart contracts.

Information in this report should be used for understanding the risk exposure of smart contracts, and as a guide to improving the security posture of smart contracts by remediating the issues that were identified.

The code is available at 0xFF1968ae4F91efCCf4d5cef823EeCd46fbe114c3 (EHMarket proxy) with implementation at 0xEEC10ab0185382b0223118ac42f82fC21B36c54D (EHMarketV3),

0x16a4f9904e222D298Ac71aA3E3Bd5C19B902C595 (Treasury Safe proxy) with implementation at 0x29fcB43b46531BcA003ddC8FCB67FFE91900C762 (SafeL2 by Gnosis),

0x071360b37F6Db934F857E6B33AD705fEee63Be14 (Vault proxy) with implementation at 0x87997081f0061BB9125D1326d2E22a8BBA7A847D (VaultV2),

0xf911C309B851Db55b97F4E2c7599a3733C2DAC69 (CashbackVault proxy) with implementation at 0x7b6A803468388E15A95A16a34A01e86f71f9ce09 (CashbackVaultV2),

0x93244b494Ca63588516ff0b3b25A09D79e9e5a3c (GovernorMultisig),

0xc7C1943BCCBc81B8dE5b7f407c00b7805eA3ea0C (Storage) in the Arbitrum One network.

The code is also available in the @evedex-official/solidity-contracts GitHub repo after 368eaec commit.

2.1 Summary

Project name	EVEDEX Arbitrum
URL	https://evedex.com/
Platform	Arbitrum Network
Language	Solidity
Centralization level	High
Centralization risk	Low

2.2 Contracts

Name	Address
EHMarketV3	0xFF1968ae4F91efCCf4d5cef823EeCd46fbe114c3
VaultV2	0x071360b37F6Db934F857E6B33AD705fEee63Be14
CashbackVaultV2	0xf911C309B851Db55b97F4E2c7599a3733C2DAC69
Treasury	0x16a4f9904e222D298Ac71aA3E3Bd5C19B902C595
Storage	0xc7C1943BCCBc81B8dE5b7f407c00b7805eA3ea0C
GovernorMultisig	0x93244b494Ca63588516ff0b3b25A09D79e9e5a3c

3. Project centralization risks

The contracts EHMarket, Vault, CashbackVault, and Storage are highly centralized. The owner can transfer out funds and update crucial parameters, that may break interaction between contracts of the project.

At the time of writing this report, ownership of these contracts has been transferred to GovernorMultisig, which does not preclude further changes in the ownership structure. We recommend conducting independent monitoring of possible ownership changes.

Ce8CR50 Governance risks

The accounts with DEFAULT_ADMIN_ROLE can withdraw all available funds.

The accounts with MATCHER_ROLE can withdraw funds within configured limits. Total amount is not limited, and funds can be drained over time.

The contract is designed to be upgradeable by the owner of the ProxyAdmin contract.

Recommendations

Recommendation

Secure all DEFAULT_ADMIN_ROLE accounts with a multisig contracts.

Establish off-chain monitoring to prevent siphoning funds by MATCHER_ROLE accounts.

CefCR55 Governance risks

The owner of the Storage contract can authorize an arbitrary address as distributor for the VaultV2 contract.

The owner can withdraw all contract's balance, including already distributed yet not withdrawn funds.

The contract is designed to be upgradeable by the owner of the ProxyAdmin contract.

Recommendations

Recommendation

Secure the ownership with a multisig contract.

CeaCR51 Governance risks

The owner can withdraw all available funds.

The owner of ProxyAdmin contract can upgrade implementation logic.

Recommendations

Recommendation

Secure the ownership with a multisig contract.

CebCR52 Governance risks

The owner is responsible for maintaining stored parameters.

Recommendations

Recommendation

Secure the ownership with a multisig contract.

CedCR53 Governance risks

Any `howManyOwnersDecide` number of owners can execute an arbitrary transaction, including ownership transfer.

Recommendations

Recommendation

Configure multisig appropriately to reduce risk of losing ownership. Increase number of independent owners.

4. Found issues



Low	7 (88%)
Info	1 (12%)

Ce8. EHMarketV3

ID	Severity	Title	Status
Ce8Iee	Low	Matcher role overrides admin role	Resolved
Ce8If7	Low	Lack of integration testing	Acknowledged
Ce8Iec	Low	Magic constants usage	Acknowledged
Ce8If0	Low	Gas optimizations	Partially fixed

Cea. CashbackVaultV2

ID	Severity	Title	Status
CeaIf4	Low	Lack of domain separator in signature verification	Acknowledged
CeaIf5	Low	The signer address can't be updated	Resolved
CeaIf3	Info	Misleading function name withdrawCrumbs	Acknowledged

Ceb. Storage

ID	Severity	Title	Status
CebIf6	Low	Lack of zero value checks	Acknowledged

5. Contracts

Ce8. EHMarketV3

Overview

A token holding contract that allows users to deposit fixed ERC20 tokens. Withdrawing is allowed only for addresses with the MATCHER_ROLE or DEFAULT_ADMIN_ROLE roles. Withdraw limits can be configured for individual users and/or for total withdrawn amount for given periods. Withdraw limits are applied only for MATCHER_ROLE, but not for DEFAULT_ADMIN_ROLE.

Issues

Ce8Iee Matcher role overrides admin role

Low Resolved

Withdrawing process is restricted to accounts with the MATCHER_ROLE and DEFAULT_ADMIN_ROLE. However, if the caller possesses both these roles, the matcher's limits are applied despite the default admin should be exempted from checks.

```
function withdrawAsset(address from, uint256 amount, uint256 requestId) external {
  bool isMatcher = hasRole(MATCHER_ROLE, _msgSender());
  require(isMatcher || hasRole(DEFAULT_ADMIN_ROLE, _msgSender()), UnauthorizedSigner());
  if (isMatcher) {
    _checkWithdrawLimits(amount, from);
    ...
  }
  ...
}
```

Ce8If7 Lack of integration testing

Low Acknowledged

The EHMarket is deployed as an upgradeable proxy and the EHMarketV3 is designed to be a proxy implementation. The contract itself is functionally tested, but there's no tests for

upgrading.

We recommend add integration testing to ensure seamless upgrade.

Ce8Iec Magic constants usage

Low Acknowledged

Request statuses are coded by using `uint8` and encoding scheme is documented by inline comments.

Solidity supports enums for better readability. Another solution to increase code readability is to use named constants for statuses.

Ce8If0 Gas optimizations

Low Partially fixed

1. Multiple reads from storage in the `getMaxWithdrawAmount()` function: `withdrawLimits.length` variable. **Update**: fixed.

2. The `getMaxWithdrawAmount()` function calls `getTotalWithdraw()` and `getUserTotalWithdraw()` functions in a cycle to calculate withdrawn amounts by hours. The withdraw limits don't overlap and sorted in ascending order, so calculations of `totalWithdrawn` and `userWithdrawn` amounts can be optimized to exclude multiple reading of the same data from storage. **Update**: fixed.

3. Multiple reads from storage in the `_checkWithdrawLimits()` function: `withdrawLimits.length` variable. **Update**: fixed.

4. Multiple reads from storage in the `_setWithdrawLimits()` function: `withdrawLimits`, `withdrawLimits.length` variables.

5. Array sorting in the `_setWithdrawLimits()` function is ineffective, it should either use in-memory sorting or be omitted completely in favor of requiring the input data to be sorted. **Update**: fixed.

Cef. VaultV2

Overview

A token holder contract that supports token distribution to arbitrary receivers. The `distribute()` function is guarded by `Storage.getBool(keccak256(abi.encodePacked("EH:PartnerVault:Distributor:", msg.sender)))`.

Cea. CashbackVaultV2

Overview

A vault contract that allows withdrawing fixed ERC20 tokens with a signed permission by back-end signer address.

Issues

Cealf4 Lack of domain separator in signature verification

Low Acknowledged

The `withdraw()` function in the CashbackVaultV2 contract uses a signature verification mechanism based on `keccak256(abi.encodePacked(...))` combined with `toEthSignedMessageHash`.

```
function withdraw(
  address recipient,
  uint80 requestId,
  uint256 amount,
  bytes memory signature
) external whenNotPaused {
  bytes32 signedMessage = keccak256(abi.encodePacked(requestId, recipient, amount));
  if (signedMessage.toEthSignedMessageHash().recover(signature) != signer) {
    revert CashbackVaultV1InvalidWithdrawSignature();
  }
```

```
    ...
}
```

The signed hash is not tied to the specific context of this contract or the blockchain it's deployed on. This lack of a domain separator creates opportunities for replay attacks.

Cealf5 The signer address can't be updated

Low Resolved

The signer address is used to authorize user's withdrawals by providing signature that includes recipient address and withdrawing amount. However, the signer can't be updated without full upgrade of the contract.

```
    /// @dev Address of signer payload for withdraw method.
    address public signer;

    /// @notice Withdraw tokens.
    /// @param recipient Tokens recipient address.
    /// @param requestId Withdraw request ID.
    /// @param amount Withdrawal token amount.
    function withdraw(
        address recipient,
        uint80 requestId,
        uint256 amount,
        bytes memory signature
    ) external whenNotPaused {
        bytes32 signedMessage = keccak256(
            abi.encodePacked(requestId, recipient, amount)
        );
        if (
            signedMessage.toEthSignedMessageHash().recover(signature) != signer
        ) {
            revert CashbackVaultV1InvalidWithdrawSignature();
        }
        if (request[requestId]) {
            revert CashbackVaultV1WithdrawAlreadyCompleted();
        }

        request[requestId] = true;
```

```
        IERC20(token).safeTransfer(recipient, amount);
        emit CashbackWithdraw(recipient, requestId, amount);
    }
```

CeaIf3 Misleading function name withdrawCrumbs ● Info ✓ Acknowledged

The `withdrawCrumbs()` function name implies that this function is for withdrawing small, leftover amounts of tokens ("dust" or "crumbs") that might be stuck in the contract.

However, its implementation is an unrestricted `onlyOwner` withdrawal function that can transfer any amount of the contract's token balance to any specified address.

Cee. Treasury

Overview

A multisig contract (Gnosis Safe proxy) with 6 owners and decision threshold of 4 at the time of writing this report.

Ceb. Storage

Overview

A contract to store project config. Supports `uint`, `int`, `bool`, `address`, `bytes`, and `string` values indexed by `bytes32` keys.

Issues

CebIf6 Lack of zero value checks ● Low ✓ Acknowledged

The contract provides separate methods for deleting data in mappings by key. However, the setter functions do not check whether the provided value is zero. This could lead to

unintended data deletion if a zero value is mistakenly passed to a setter.We recommend adding checks to prevent zero values from being set, in order to avoid accidental data deletion.

Ced. GovernorMultisig

Overview

A multisig contract based on Multiownable contract. Supports batched arbitrary calls guarded with threshold decision of owners.

6. Conclusion

7 low severity issues were found during the audit. 2 low issues were resolved in the update. The reviewed contracts are highly dependent on the owner's account. See the centralization risks chapter. This audit includes recommendations on code improvement and the prevention of potential attacks.

Appendix A. Issues' severity classification

- **Critical.** Issues that may cause an unlimited loss of funds or entirely break the contract workflow. Malicious code (including malicious modification of libraries) is also treated as a critical severity issue. These issues must be fixed before deployments or fixed in already running projects as soon as possible.
- **High.** Issues that may lead to a limited loss of funds, break interaction with users, or other contracts under specific conditions. Also, issues in a smart contract, that allow a privileged account the ability to steal or block other users' funds.
- **Medium.** Issues that do not lead to a loss of funds directly, but break the contract logic. May lead to failures in contracts operation.
- **Low.** Issues that are of a non-optimal code character, for instance, gas optimization tips, unused variables, errors in messages.
- **Informational.** Issues that do not impact the contract operation. Usually, informational severity issues are related to code best practices, e.g. style guide.

Appendix B. Issue status description

Resolved. The issue has been completely fixed.

Partially fixed. Parts of the issue have been fixed but the issue is not completely resolved.

Acknowledged. The team has been notified of the issue, no action has been taken.

Open. The issue remains unresolved.

Appendix C. List of examined issue types

- Business logic overview
- Functionality checks
- Following best practices
- Access control and authorization
- Reentrancy attacks
- Front-run attacks
- DoS with (unexpected) revert
- DoS with block gas limit
- Transaction-ordering dependence
- ERC/BEP and other standards violation
- Unchecked math
- Implicit visibility levels
- Excessive gas usage
- Timestamp dependence
- Forcibly sending ether to a contract
- Weak sources of randomness
- Shadowing state variables
- Usage of deprecated code

Appendix D. Centralization risks classification

Centralization level

- **High.** The project owners can manipulate user's funds, lock user's funds on their will (reversible or irreversible), or maliciously update contracts parameters or bytecode.
- **Medium.** The project owners can modify contract's parameters to break some functions of the project contract or contracts, but user's funds remain withdrawable.
- **Low.** The contract is trustless or its governance functions are safe against a malicious owner.

Centralization risk

- **High.** Lost ownership over the project contract or contracts may result in user's losses. Contract's ownership belongs to EOA or EOAs, and their security model is unknown or out of scope.
- **Medium.** Contract's ownership is transferred to a contract with not industry-accepted parameters, or to a contract without an audit. Also includes EOA with a documented security model, which is out of scope.
- **Low.** Contract's ownership is transferred to a well-known or audited contract with industry-accepted parameters.

 contact@hashex.org

 @hashex_manager

 blog.hashex.org

 linkedin

 github

 twitter

#HashEx
BLOCKCHAIN SECURITY